从特征供给瓶颈，到自适应在线训练

核心问题：特征不能全量驻留时，怎样让 GPU 持续获得训练数据？

Vanilla

按需读取

避免全量物化

→

v1

单进程优化

去重与复用

→

v2

多进程升级

采样 / 组装 / 训练分离

汇报顺序

问题 → v1 → 新瓶颈 → v2 架构与算法 → 实验 → 耗时图 → RSS 图

优化对象不是数据集名称，而是重复率、访问成本、工作集大小和资源压力。

按需读取省下内存，却把特征生产放进训练循环

Vanilla 已有基础 SQL / raw cache；问题是重复生产和串行供给仍然昂贵。

采样节点
同一节点可能重复出现

→

SQL + 解码
重复访问与数据转换

→

文本 + Frame
重复编码与组装

→

GPU 更新
等待 ready 数据

问题 A ｜重复工作

同一节点、同一文本被多次处理；

原始行缓存不等于最终特征复用。

问题 B ｜供给不足

SQL 、编码和组装阻塞训练；

端到端耗时还包含首次初始化。

先减少每个 batch 的重复工作，再提高特征生产的并行度。

把重复请求合并，只对缺失部分生产特征

三层复用：节点请求、文本向量、已编码特征。

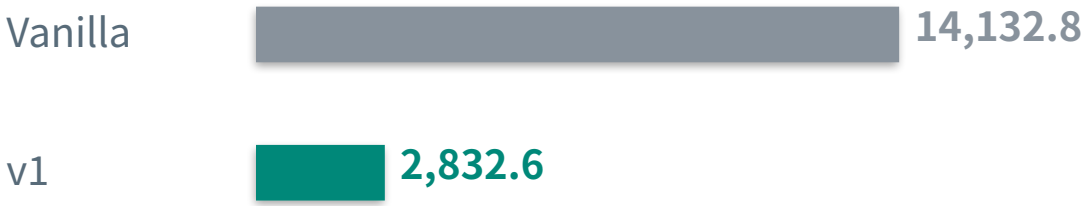
改动	省掉什么	必须保留什么
节点去重 + 跨 batch 窗口	重复 SQL / 解码 / 编码	恢复每个 batch 的节点位置和顺序
稀疏 ID 查询 / 稠密块读取	无效扫描和过量取数	相同请求行与时间约束
raw → text → encoded 缓存	逐步省掉后续转换工作	仅缓存静态特征、冻结文本向量
direct 文本路径 + 有界线程流水线	通用调度开销，重叠 SQL 与编码	兼容回退；限制队列与工作集

v1 的主要收益来自“少读、少转、少编码”，不改变训练输入的数量。

重复结构不同，同一缓存策略的收益就不同

历史单轮结果： Salt 的节点复用高， Stack 的节点复用低。

Salt： Vanilla → v1



4.99× • 秒

节点重复因子 2.114

raw 行覆盖率 98.70%

Stack： Vanilla → v1



1.22× • 秒

节点重复因子 1.011

raw 行覆盖率 4.86%

重复率决定“能省多少”；缓存查询和复制成本决定“省下来是否划算”。

assemble 成为瓶颈：单进程优化后仍供给不足

先分离采样、组装与训练，再处理多进程缓存的访问成本。

高命中仍然慢

HM / Salt

整块 RPC 、共享锁、高频查表

工作集超出内存

Arxiv / Stack / Trial

缓存填满后反复查询、跨轮重算

并发资源持续增长

Stack / Amazon

mmap 数量、 WAL 、刷盘等待

准备工作压倒训练

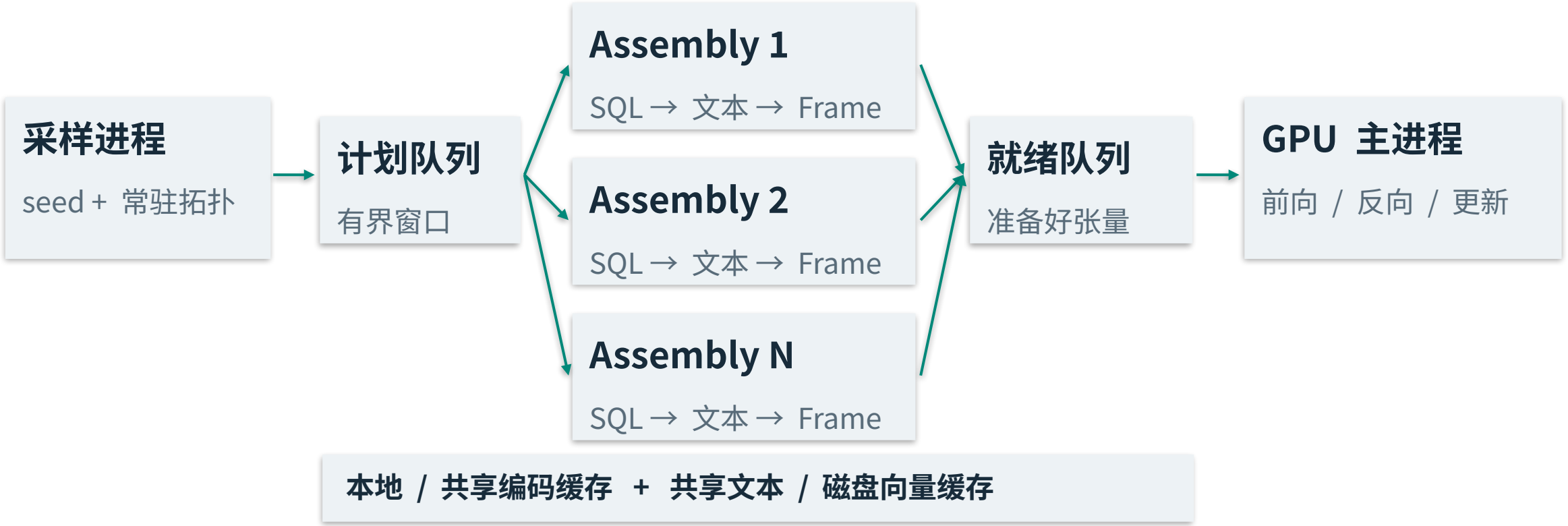
Event / F1

宽表冷统计；小任务启动成本

v2 需要同时解决：供给并行度、命中成本、缓存容量和长期运行资源。

采样、特征组装、 GPU 训练分进程执行

v1 单进程优化后， assemble 供给仍受限； v2 将三个阶段拆分并重叠执行。



队列与在途窗口受限； ready 顺序减轻队头等待，但会改变 batch 更新顺序。

根据行数与内存预算，自动合并数值列扫描

不按 Event 特判，不抽样；精确统计结果保留。

1 估算单列缓冲：行数 × 8 字节
2 分组列数 = min(32, 128 MiB / 单列大小)
3 合并读取，批次最多 8,192 行
4 内存 partition 选取精确分位点
5 单列超预算 → 回退流式 SQL 路径

Event： 完整冷初始化





3.57× • 秒

Schema： 1,895.62 → 335.66 秒；图拓扑： 267.95 → 269.92 秒。

329 项统计核对：分位数完全一致；均值 / 标准差最大绝对差约 8×10^{-13} 。

总初始化减少 72.0%（3.57×）；剩余约 44.5% 时间在图拓扑构建。

采样输出计划，组装只处理唯一物理行

保持时序邻域与逻辑节点实例；合并的是静态特征请求。

采样计划
节点 / 边 / 时间

→

窗口去重
唯一行 + 逆映射

→

读取与编码
只生产必要特征

→

恢复批次
逻辑位置保持不变

访问特征	Salt	Ratebeer
历史请求 / 唯一行	2.114	1.105
理论可消除重复行工作	52.7%	9.5%

v1 建立窗口复用； v2 将采样与 assemble 分离，让计划生产与特征生产重叠。

重复命中时，只查本地槽位索引

向量仍共享；本地仅保存“文本 → 已发布槽位”，避免复制整份向量。

观察窗口

64 次调用

≥1,024 次请求

→

自动激活

共享命中 ≥ 80%

查询累计 ≥ 5 ms

→

本地命中

直接批量 gather

跳过共享哈希与锁

自动退出

连续两个窗口本地命中 <20%：

释放索引，并进入冷却期。

18,260 → 5,208 s

Salt 两轮完整耗时；模型输入不变

共享查询由约 30,532 降至 802 worker 秒

高命中率任务的重点是降低命中成本，而非继续增加编码能力。

缓存放在哪里，要看访问成本与额外复用

HM 的高复用与 Ratebeer 的低复用，都可能被跨进程访问成本限制。

机制 / 配置	HM： 历史同预算诊断	Ratebeer： 本次同预算诊断
共同预算	总计 2 GiB	总计 2 GiB， 4 worker
共享命中路径	整块 → 只传请求行	查询 + 发布仍昂贵
本地布局	减少 RPC 与重建	每 worker 512 MiB
观测结果	206.0 → 157.7 秒 / 1000 批	1104.6 → 980.2 ms/batch
主要取舍	跨进程复用与传输量	命中率 16.5% → 1.6%

判据：共享多省下的编码成本，能否覆盖多付出的查询、发布与通信成本?

Ratebeer 流水线平均每批减少 11.26%；这是显式配置选择，尚非自动部署。

把跨轮可复用的 embedding 留在磁盘

本地索引 → 共享文本缓存 → 磁盘缓存 → 真正缺失才编码。

本地槽位
命中即返回

→

共享向量
命中即返回

→

磁盘向量
命中即返回

→

GloVe 编码
发布结果并复用

新缓存的启用条件

数据库 ⩾ 256 MiB； 模型输入 ⩾ 4,096；

模型耗时 ⩾ 1 s； 内存命中率 ⩽ 10% 。

已有兼容磁盘缓存可直接复用。

观察到的收益

Arxiv： 第二轮编码显著减少。

Trial： 热缓存复测两轮编码均为 0 。

F1： 小数据集不激活磁盘路径。

冷构建要付编码与写入成本；热缓存成绩必须与冷启动成绩分开。

向量已缓存，物理碎片仍会拖慢 assemble

把布局整理融入持久化方案：先复用计算结果，再减少命中后的文件管理。

原布局： 79,587 片

映射上限 1,024 / worker

→

离线合并 + 校验

2,891,661 条向量逐字节一致

→

新布局： 353 片

有效集合可留在映射缓存

Stack： 同逻辑缓存， 200 批 / 次	原布局	合并后
映射打开次数 / 次运行	238,288	1,312.5
映射淘汰次数 / 次运行	234,192	0
流水线平均 ms/batch	428.6	370.6

在线每批减少 13.52%；离线整理 144.43 秒另计，不覆盖原缓存。

缓存能命中，还要能长期稳定运行

缩短临界区、限制映射数量，并随读写阶段调整日志回收。

Stack ｜查询与映射

- 哈希、 gather 移到锁外，批量发布。
- mmap 最多保留 1,024 个， LRU 回收。
- 共享查询短测： 296.0 → 53.5 秒。
- 完整两轮： 3,665.66 秒。

Amazon ｜ WAL 与写入

- 每 writer 追加 8,192 行固定分片。
- 预算参考索引大小和剩余磁盘。
- 读多写少时降低 WAL 回收阈值。
- 避免固定小阈值造成频繁刷盘。

WAL 阈值是回收触发条件，不是硬上限； Amazon 完整成绩未包含后加读阶段策略。

少做一个操作，不一定让一批更快

同一套验收原则覆盖已保留的优化和已撤回的候选。

尝试	局部观察	端到端结果	当前处理
CPU 行选择	Salt 展开 − 17.03%	91.58 → 88.63 秒	保留；短测 − 3.22%
融合多块缓存投影	查询 − 18.90%	128.32 → 129.30 秒	候选归档
编码缓存前移到 SQL 前	Avito SQL 行数 − 81.56%	共享路径仅 − 0.17%	候选归档
同前移 / Ratebeer 本地	进一步省去部分读取	仅再减少 0.87%	恢复原读取路径

行选择试验与本轮 Ratebeer / Stack 使用不同窗口和基线，百分比不能相加。

默认方案依据完整工作窗口的实际时间，不能依据累计 worker 时间单独决定。

Ratebeer： 先修复发布竞争，再评价吞吐

并发修复解决数据损坏，不应直接包装成性能加速算法。

旧发布顺序

索引可见 → RPC 搬迁 storage

并发读可能访问旧地址

新发布顺序

先完成共享 storage → 再发布索引

后续序列化不搬迁已发布对象

验证：独立协调复现 + 回归测试 + 两轮各 10,000 batch 完成。

保留失败输入、精确 batch key 和类别校验；不截断坏值、不跳过样本。

仍有大量文本编码、 SQL 和组装开销，磁盘策略未启用。

首个 CUDA 异常未保存输入，不能确认与后续已复现竞争完全同源。

把新旧优化放在同一套选择逻辑中

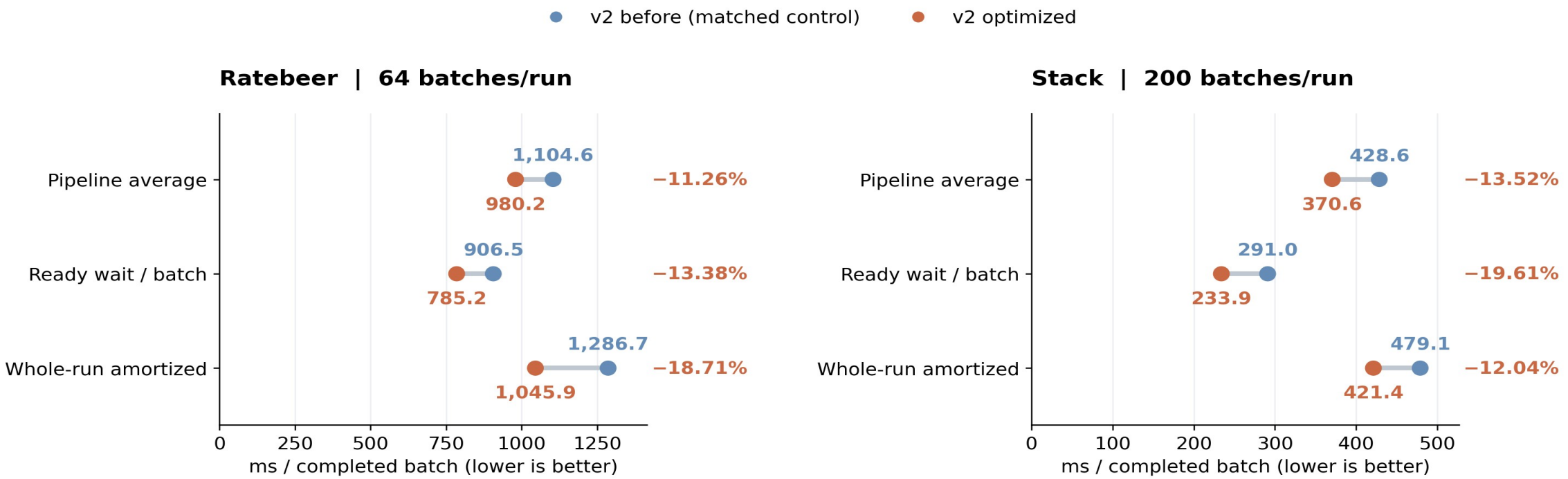
数据集只是代表证据；同一任务可同时具备多种特征。

观察信号	v2 对应机制	证据 / 实现边界
宽表冷统计重	预算分组 + 精确分位数	Event 冷初始化 3.57×
重复文本多、共享查询贵	本地槽位索引 + 共享向量	Salt 完整修复记录
编码行命中传输贵	请求行投影 + 共享 / 本地配置	HM； Ratebeer 本地每批 − 11.26%
文本工作集大	有条件持久化冻结向量	区分首次创建与热复用
持久化小分片过多	离线合并 + 映射预算筛选	Stack 每批 − 13.52% ，整理另计
训练批次少	复用离线产物、减少固定开销	F1；少 worker 仍待对照

自动机制、显式配置和未来方案分开描述；不把候选写成已部署能力。

新的每 batch 结果：供给时间与等待一起下降

同请求、同预算；每种配置重复两次，分别完成 64 / 200 批。

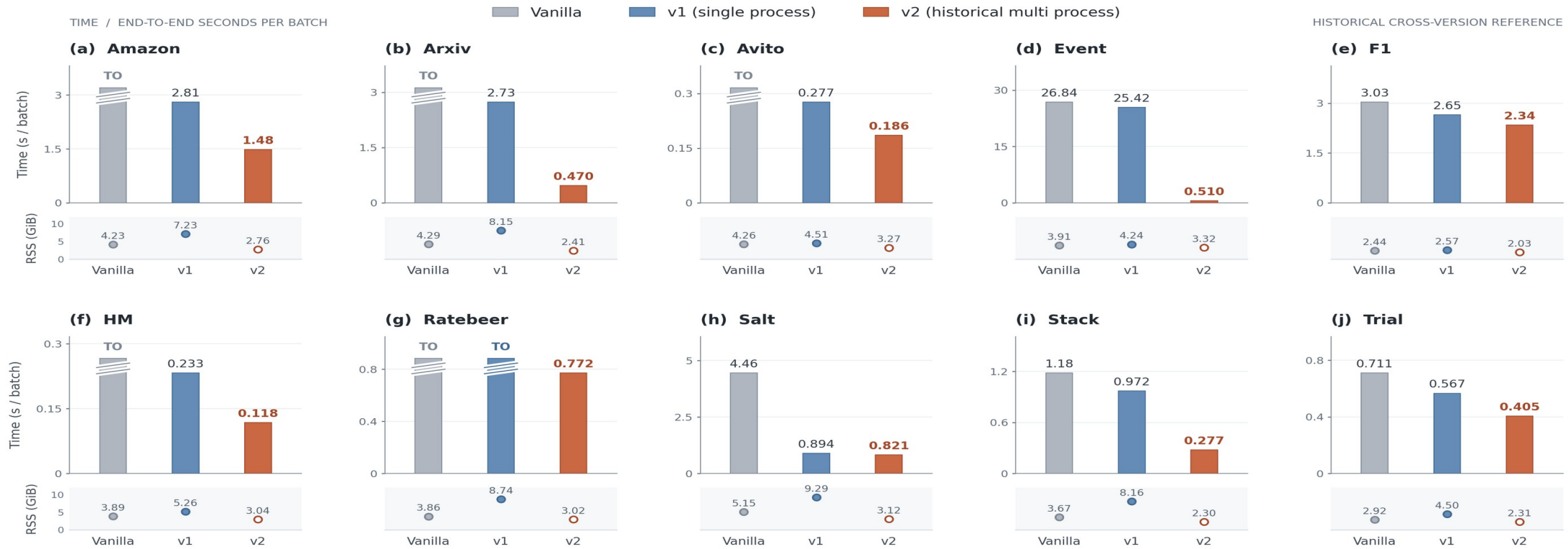


主指标：流水线总时间 / 完成批数；包含内部启动与排空，不是单批独立延迟。

Ratebeer −11.26% | Stack −13.52%； Stack 一次性整理成本为 144.43 秒。

跨版本概览，与本轮 v2 对照分层展示

上方是历史端到端时间与 RSS；下方是本轮配对流水线结果。



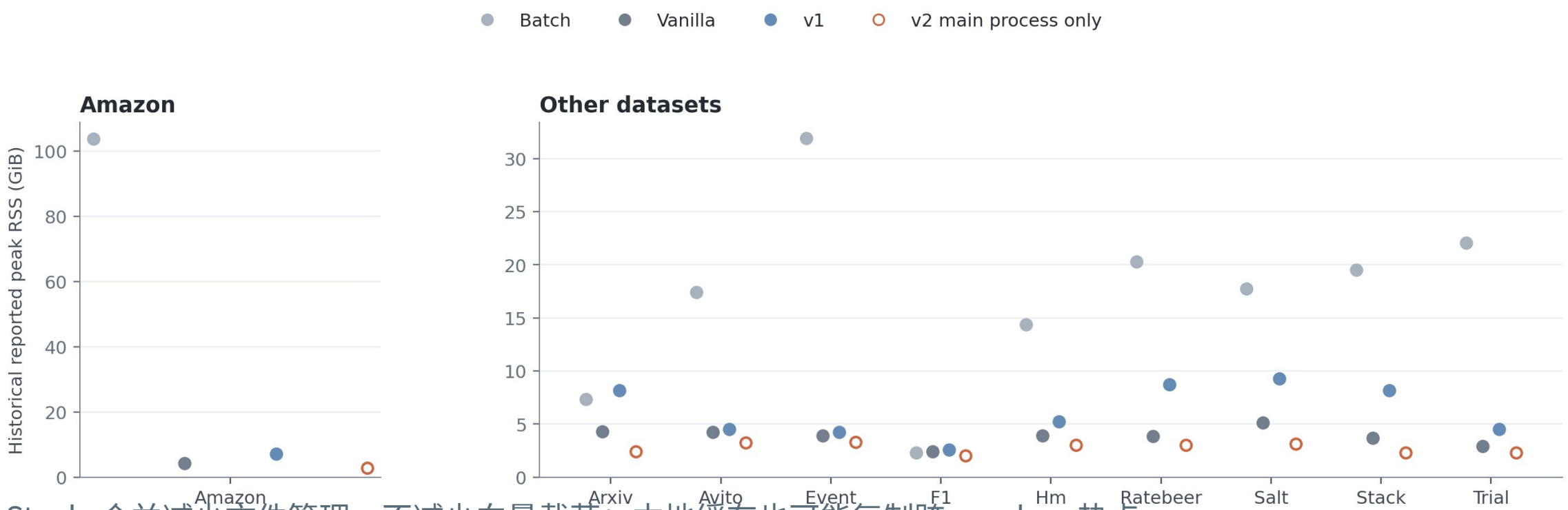
CURRENT MATCHED v2 EXPERIMENTS | pipeline ms / batch

Ratebeer: 1,104.6 → 980.2 (−11.26%; 64 batches/run)　Stack: 428.6 → 370.6 (−13.52%; 200 batches/run)

Two repeats per variant; these short-run pipeline measurements do not replace the historical end-to-end bars or RSS.

新优化测了吞吐，没有产生新的作业内存结论

Batch 、 Vanilla 、 v1 为历史报告； v2 空心点仅为主进程 RSS 。



Stack 合并减少文件管理，不减少向量载荷；本地缓存也可能复制跨 worker 热点。

RSS 未更新测量；不再用主进程记录值推导“整个作业内存降低多少倍”。

减少常驻特征，还需要覆盖整个进程树

缓存预算、队列上限与主进程 RSS ，都不等于系统总内存。

训练主进程

模型 / 优化器 / 当前 batch

采样 + manager

拓扑 / 索引 / 共享缓存

Assembly × N

GloVe / SQL / 临时 Frame

测量补齐：进程树 PSS / USS 、 cgroup 峰值、显存与磁盘资源。

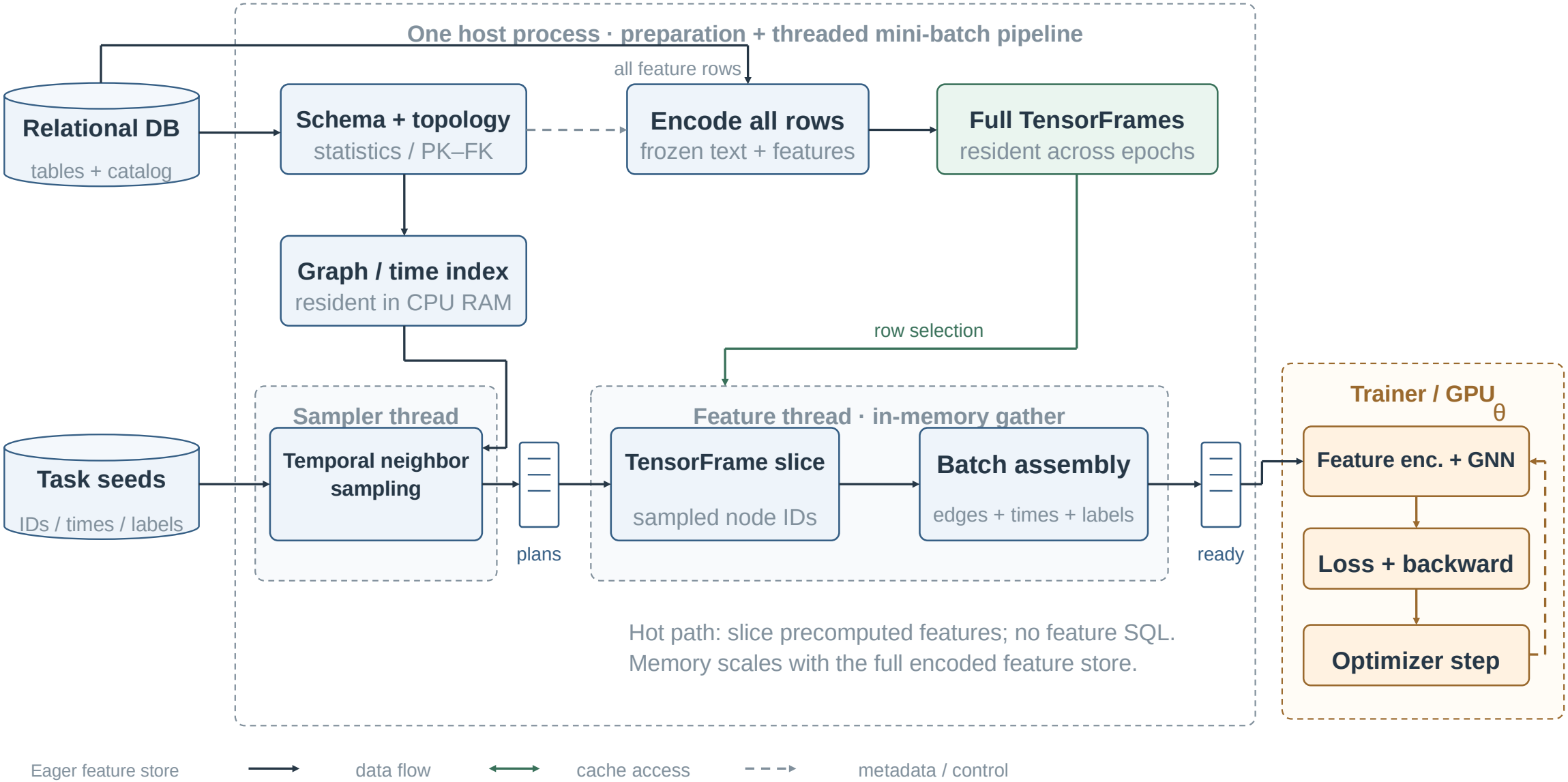
性能优先级：验证 Ratebeer 长窗口；补 Amazon 诊断；优化 Event 拓扑。

最终总评：固定代码和资源预算，分别做冷缓存与热缓存十库复测。

主线不变：少做重复工作 → 提高供给 → 让命中更便宜 → 控制资源与冷启动。

(a) Batch ｜全量物化训练架构

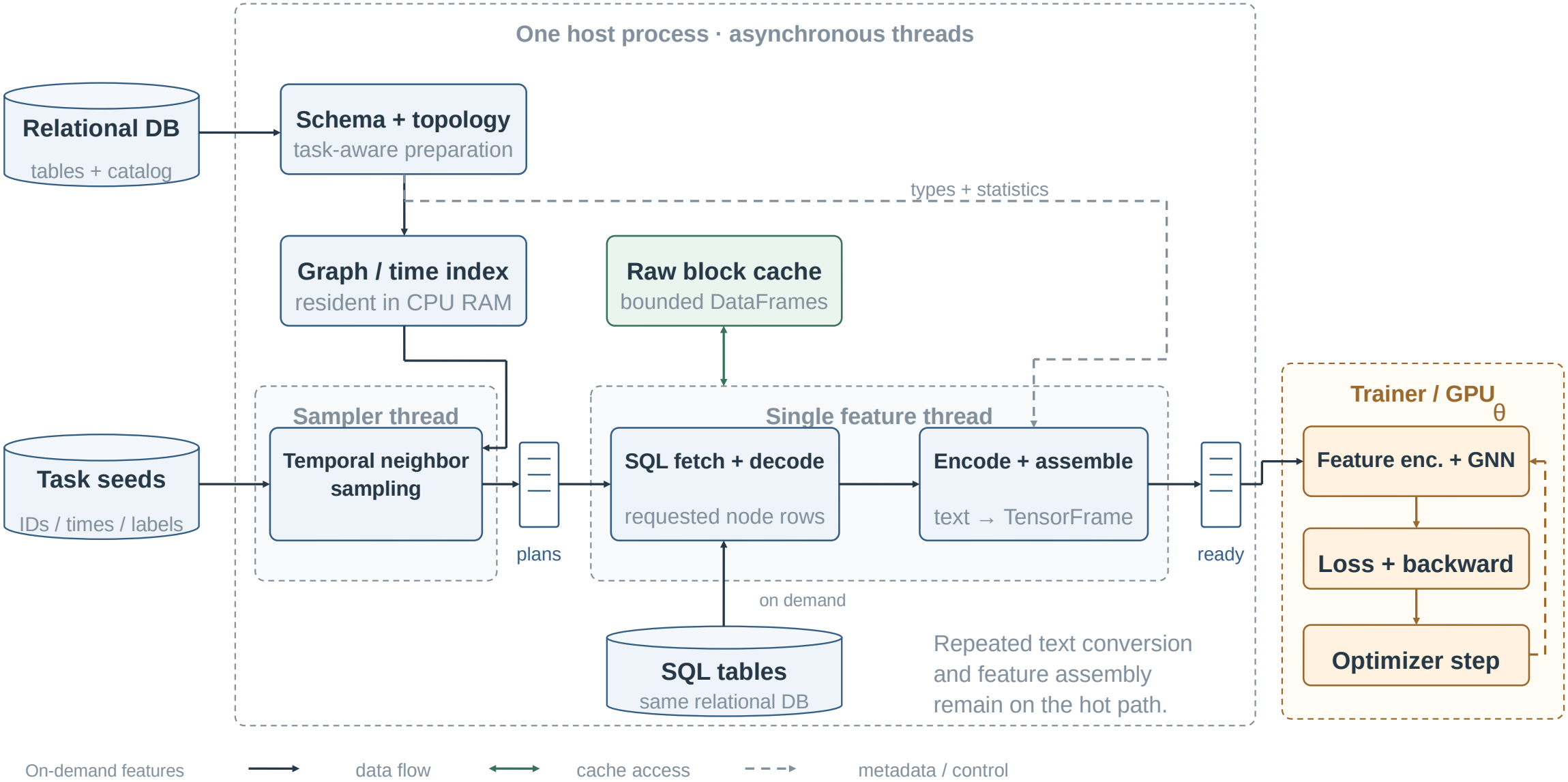
训练前生成全部特征；训练时从常驻 TensorFrame 切片。虚线框表示执行范围。



Batch: eager materialization followed by sampled mini-batch training. Static features are resident; model parameters remain trainable.

(b) Vanilla ｜按需读取的在线训练架构

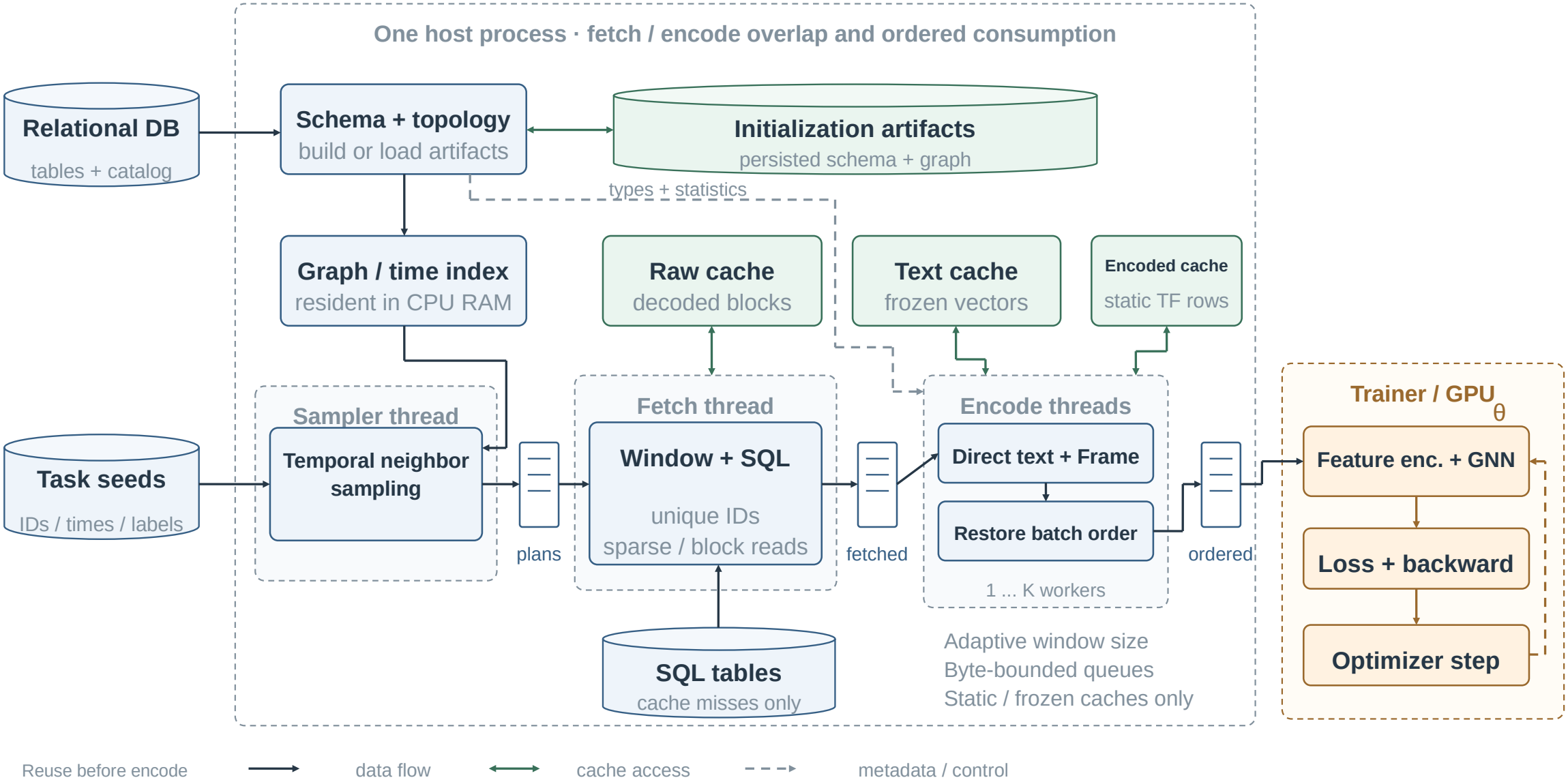
单进程内异步采样、特征线程与训练；已有 raw cache ，但缺少后续文本与编码特征复用。



Vanilla: bounded online feature access with a raw cache and asynchronous threads; static text / encoded-feature reuse is not yet added.

(c) v1 ｜单进程优化：窗口复用与分层缓存

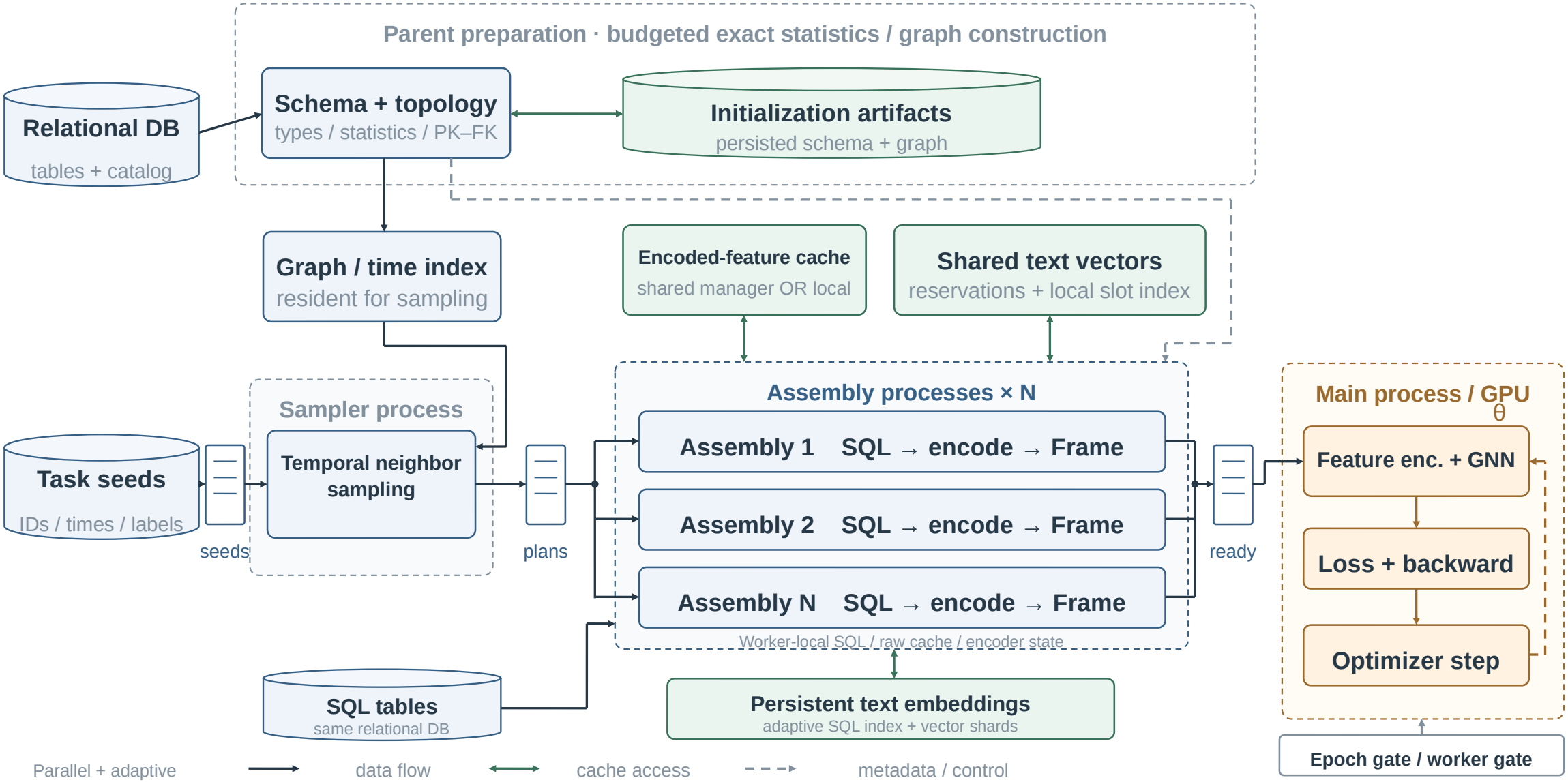
在单进程线程流水线中合并请求，并复用 raw 、 text 与 encoded 特征；训练按序消费。



v1: deduplicated feature windows, layered in-process caches, overlapping fetch / encoding threads, and restored per-batch inputs.

(d) v2 | 多进程与自适应缓存训练架构

assemble 成为供给瓶颈后， v2 将采样、组装、训练分进程执行。



v2 upgrades v1 after assembly becomes a bottleneck: sampling, assembly and training run in separate processes connected by bounded queues.